

Wake · sleep · long-term memory in-fra 는 어떻게 나누는가

어려운 Study paper 를 그림과 예로 다시 읽는 안내서

읽는 법 이 문서는 원 Study 의 결론을 바꾸지 않고, 질문→세 문장→직관→예/반례→그림→대안→미증명 지점 순서로 풀어 쓴다. 파란색 training 용어는 별도 Background paper 의 해당 개념으로 연결된다. 주장 ID 와 figure ID 는 원 Study 와 동일하다.

Contents

E10 · Wake·sleep·long-term memory infra 는 어떻게나뉘는가	2
이 권이 답할질문	2
먼저 세 문장	2
직관	2
예와 반례	3
그림 읽기	4
대안과 비교	4
아직 모르는것	6
더 깊이 읽기	6
문서 경계와 사용법	7

E10 · Wake·sleep·long-term memory infra 는 어떻게나눠는가

이 권이 답할질문

배포 후 학습이 wake 와 sleep 으로 나뉜다면 cluster 와 data plane 도 나뉘어야하는가? 그렇다. 다만 세 개의 독립 silo 를 만드는 것이 아니라 wake SLA, background training, durable memory 가 서로 다른 최적점을 가지므로 **control plane** 과 **versioned state contract** 로 연결해야 한다. Infra 목표는 state movement 를 줄이고, sleep output 을 안전하게 publish 하며, 외부 memory 와 model delta 를 같은 lineage 로 관리하는 것이다.

이 권은 system infrastructure 를 설명한다. 근거는 **STC-C025 · author_claim · confidence high**, **STC-C040 · synthesis · confidence medium**, **STC-C046 · synthesis · confidence high**, **STC-C053 · hypothesis · confidence medium**, **STC-C055 · hypothesis · confidence medium** 다.

먼저 세 문장

1. Wake cluster 는 TTFT·ITL·availability 를, sleep cluster 는 throughput·sample efficiency·batching 을 최적화한다. 같은 GPU 를 써도 scheduler objective 가 다르다.
2. Long-term memory fabric 은 episode, vector, graph, recurrent state, adapter, optimizer snapshot 을 tiering 하고 provenance·version·delete lineage 를 유지한다.
3. Wake 와 sleep 사이의 핵심 interface 는 raw log dump 가 아니라 **immutable evidence ID + state generation + compatibility/version metadata** 다.

직관

Wake cluster

Wake path 는 사용자가 기다린다. Model weights, active adapter, KV/recurrent state, hot retrieval index 가 low-latency tier 에 있어야한다. Online update 를 허용하면 inference 만이 아니라 read-modify-write 가 생긴다. 하지만 긴 backward 나 validation 을 response path 에 넣으면 tail latency 가 무너진다.

Sleep cluster

Sleep path 는 여러 user/session 의 job 을 batch 하고 cheap/off-peak accelerator 를 쓸 수 있다. Teacher rollout, embedding, graph compaction, LoRA training, regression evaluation 이 섞인다. High throughput 이 중요하지만 한 job 의 잘못된 output 이 많은 future response 에 영향을 주므로 isolation 과 reproducibility 가 필요하다.

Long-term memory fabric

Memory medium 마다 access pattern 이 다르다.

- Episode log: append-heavy, immutable, cold-to-warm scan
- Vector index: embedding read, incremental insert, compaction
- Temporal graph: small random read/write, lineage traversal
- Recurrent/neural state: dense tensor snapshot, prefix keying
- Adapter: versioned blob, read-mostly serving, occasional write

- Optimizer/checkpoint: large sequential write/read, sleep job scoped

하나의 HBM 에 모두 넣는 것은 비경제적이다. Hot state 만 accelerator near-memory 에 두고, warm metadata·index 와 cold evidence 를 tiering 해야한다.

Control plane

버전관리은 “model v3” 하나로 끝나지 않는다. Base model, tokenizer, adapter, memory schema, index build, graph snapshot, policy, evaluator version 의 compatible tuple 이 필요하다. 롤백은이 tuple 의 active pointer 를 이전 generation 으로 원자적으로 돌리는일이다.

Recurrent state cache

Full attention serving 은 token prefix 가 같으면 KV prefix cache 를재사용한다. RNN/HOPE/Titans-like model 은 prefix 를 압축한 state 를재사용해야 한다. **STC-C025** · author_claim · confidence high 가지적하듯 state 를 materialize·key·share 하지 않으면 architecture 의 linear-time 장점이 serving 에서 사라질 수 있다. Cache key 에는 model/adapter version, exact prefix hash, state schema 가 들어가야 한다.

시스템 모델링 관점

State migration byte 가 compute FLOP 보다 큰 구간이 있다. Sleep job 이 adapter 를 만들기 위해 remote episode 와 checkpoint 를 모두 GPU 로 옮기면 near-data preprocessing 의 가치가 커진다.

예와 반례

한 sleep generation 의 publish protocol

1. Control plane 이 evidence snapshot ID 와 policy/evaluator version 을고정한다.
2. Job scheduler 가 graph/index job 과 GPU training job 을 DAG 로 실행한다.
3. Output 을 new generation namespace 에 쓰고 old generation 을 건드리지않는다.
4. Regression, deletion, poisoning, cost gate 를 실행한다.
5. 통과하면 active pointer 를 atomic switch 한다.
6. Canary metric 이 나빠지면 이전 pointer 로 즉시 rollback 한다.
7. Retention period 뒤에 unreachable generation 을 garbage collect 한다.

이 protocol 은 database compaction 과 model deployment 를 결합한 형태다.

Data movement 를 줄이는방법

- Episode 와 graph close-to-storage 에서 dedup·filter 해 accelerator 로보내는 token 을 줄인다.
- Base model 은 sleep GPU 에 상주시키고 user/domain adapter job 만이동한다.
- Adapter snapshot 은 copy-on-write 로 만들고 unchanged block 을 공유한다.
- Recurrent state 는 wake region 에서 reusable cache 로 materialize 하고 remote migration 을 최소화한다.
- Evaluation dataset 과 teacher logits 를 content-addressed cache 로재사용한다.

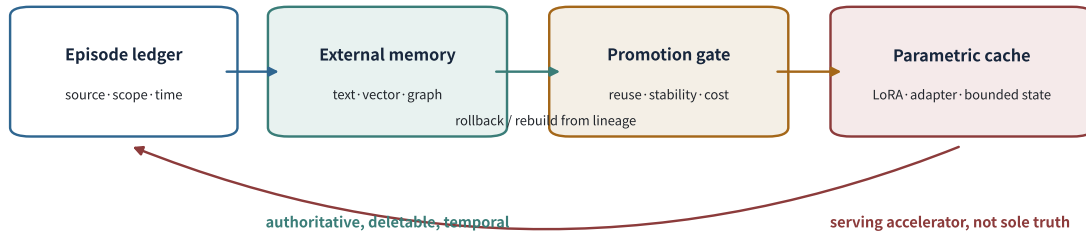
Hybrid: external system-of-record → selective reversible promotion

Figure 1: STC-F006. Hybrid system-of-record and selective-promotion architecture. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 — Wake 와 sleep 이 같은 mutable weight 를 동시에쓰

Inference 가 읽는 중 training 이 같은 weight 를 갱신하면 reproducibility 와 batching 이 깨진다. Mutable in-place update 대신 generation 을 분리하고 publish boundary 에서 pointer 를 바꿔야 한다.

반례 — 모든 user state 를 HBM 예상주

Active user 분포는 long tail 이다. HBM capacity 를 늘려도 cold state 가 hot state 를 밀어내면 효율이 낮다. Access frequency, state size, regeneration cost 로 tier 를 정해야 한다.

그림 읽기

Architecture 의 중심은 external source-of-record 와 selective promotion 이다. Wake cluster 는 derived state 를 읽고, sleep cluster 는 evidence 에서 새 derived state 를 만든다. 화살표는 one-way copy 가 아니라 provenance link 와 invalidation path 를 뜻한다.

State-migration roofline 은 x 축 byte movement 와 y 축 compute 를 놓는다. Graph/index compaction 은 bandwidth·random write bound, adapter training 은 compute/HBM bound, checkpoint publish 는 sequential bandwidth bound 일 수 있다. Device solution 은 workload 를 한 숫자 “AI bandwidth” 로 묶지 말고 phase 별 operational intensity 를 봐야 한다.

Failure control 은 control plane 의 요구사항이다. Poisoned evidence 를 걸러도 derived adapter 가 이미 publish 됐다면 lineage 를 따라 invalidation 하고 rebuild 해야 한다. Observability 에는 answer metric 뿐 아니라 state generation, evidence coverage, rollback latency 가 포함된다.

대안과 비교

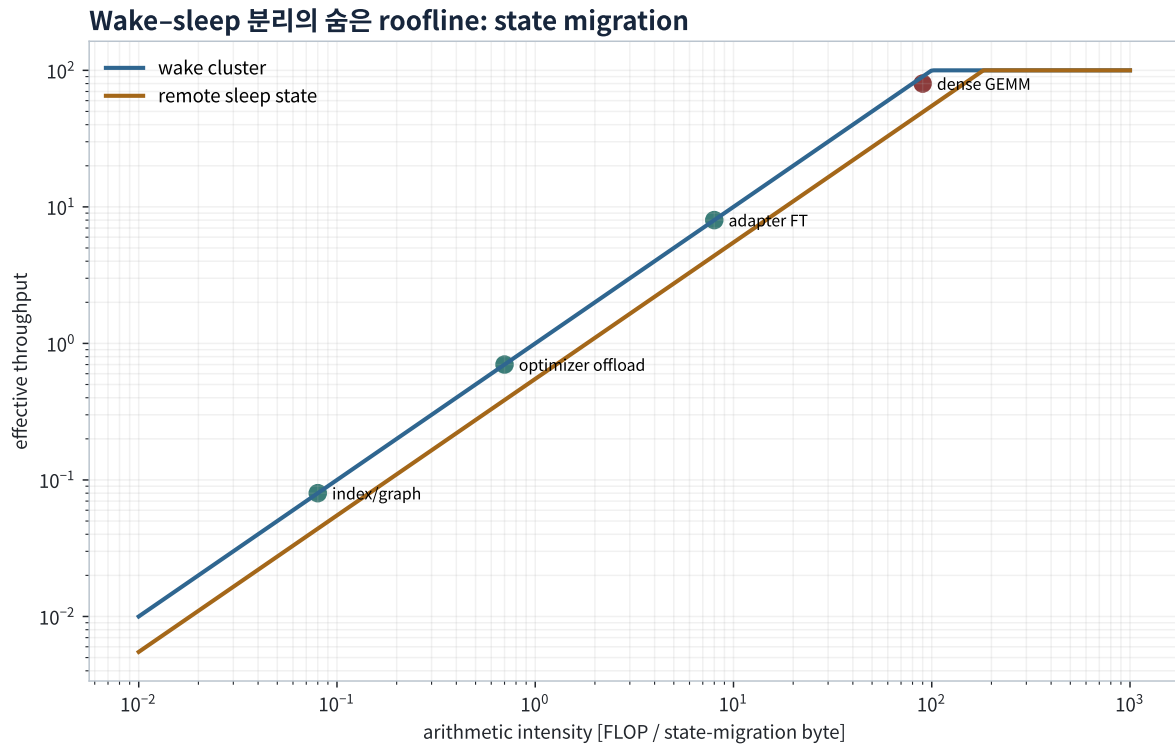


Figure 2: STC-F011. State-migration roofline for wake and sleep clusters. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

Failure mode마다 lifecycle control을 붙인다

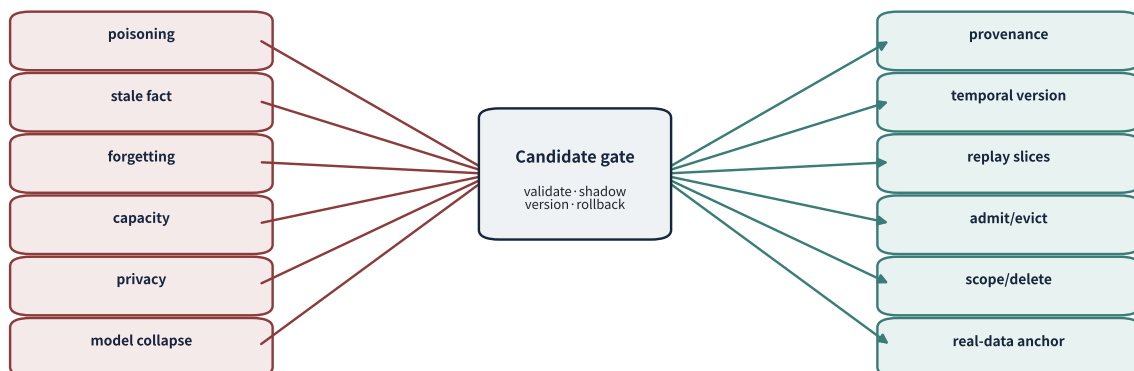


Figure 3: STC-F013. Failure modes and lifecycle controls. 원 Study 의 저자작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

plane	SLO	hot state	주 병목	필수 primitive
wake inference	TTFT/ITL, availability	weights, KV/state, hot index	HBM/BW, batching	cache, routing
wake learning	bounded latency	fast state, accumulator	RMW, interference	isolated delta
sleep compute	throughput, utility	training activations, batch data	GPU compute/data feed	DAG, checkpoint
memory fabric	durability, lookup	tier 별 working set	random write/migration	index, tiering
control plane	correctness	metadata/lineage	version explosion	atomic publish/rollback

샤딩은 user 나 entity 를 분산하지만 cross-user/domain consolidation 이 필요할 때 shuffle 이 생긴다. Reuse 와 privacy boundary 가 같은 shard key 가 되도록 설계하면 이동을 줄일 수 있다.

아직 모르는것

- Wake/sleep cluster 를 물리 분리할지 elastic shared pool 로 운영할지 workload evidence 가 부족하다.
- Recurrent state cache 의 real trace hit ratio 와 eviction policy 가 공개되지 않았다.
- Adapter·graph·vector 를 하나의 transaction 으로 publish 하는 표준이 없다.
- Sleep job 의 queue delay 와 memory staleness 가 user utility 에 미치는 관계가 측정되지 않았다.
- Delete request 가 derived weight 까지 전파되는 rebuild SLA 와 비용이 불명확하다.

핵심

STC infra 의 핵심 제품은 “밤에 쓰는 GPU” 가 아니라 evidence 에서 derived state 를 만들고, 최소 이동으로 검증·publish·rollback 하는 lifecycle fabric 이다.

더 깊이 읽기

- 원 분석: wake/sleep/memory/control plane architecture
- Training Background: 샤딩, 버전관리, 롤백, 기억라우팅, 관측가능성, 서빙산출물
- Claim route: **STC-C025** · author_claim · confidence high · **STC-C040** · synthesis · confidence medium · **STC-C046** · synthesis · confidence high · **STC-C053** · hypothesis · confidence medium · **STC-C055** · hypothesis · confidence medium

문서 경계와 사용법

이 문서는 쉬운 설명본이지 새로운 증거원이 아니다. 수치 · 인용 · claim status 는 원 Study paper 와 claim map 이 우선한다. 학습 개념은 Training Background paper 를 함께 읽고, 논문 재현이나 외부 배포 판단에는 고정 source 와 rights audit 를 사용한다.